Smart Plant System: Technical Report

CSC635 Internet of Things

Instructor: Dr. Ayman Tajjedine
ayman.tajeddine@lau.edu.lb

Hasan Ramadan
hasan.ramadan03@lau.edu

Marc Alwan
marc.alwan@lau.edu

Department of Arts and Sciences
Lebanese American University
Beirut, Lebanon

April 2026

1 Problem Definition and Use Case

Modern plant care, especially in homesteading environments, relies heavily on manual monitoring and intervention. People that own plants or a small agricultural area, must regularly check soil moisture, temperature, light exposure, and water availability to ensure healthy growth and other responsabilities that are most probably not their priority since it is not their main job, it is more of a hobby or side-job. This approach presents several problems:

- Inconsistency in care due to human error or forgetfulness
- Lack of real-time monitoring, leading to delayed responses to critical conditions
- Inefficient water usage, either overwatering or underwatering plants
- No remote access, making it difficult to manage plants when away

These limitations can negatively impact plant health, reduce yield (in agricultural contexts), and increase maintenance effort.

1.1 Proposed Solution

This project introduces a Smart Plant Monitoring and Automation System built using an ESP32, environmental sensors, actuators, and a cloud-connected backend.

The system continuously monitors environmental conditions and automatically controls actuators such as:

- water pump
- fan
- servo-controlled cover
- stepper motor for targeted irrigation

Additionally, the system supports remote control via a web application, allowing users to override automatic behavior and customize thresholds.

1.2 Use Case

1.2.1 Primary: Automated Plant Care

A user installs the system in a plant setup (e.g., indoor garden or greenhouse). The system performs the following:

1. Monitoring (collecting data using sensors)
2. Decision Making (based on certain environment and resources conditions
3. Actuation (like watering or alerts)

1.2.2 Secondary: Remote user control

Using a web application:
User can:

1. Turn actuators ON/OFF manually
2. Enable/disable automatic mode
3. Adjust thresholds (e.g., soil moisture, temperature)

System logs:

1. Sensor readings
2. Actuator states
3. Source of action (automatic vs manual)

1.2.3 Tertiary: Data Logging & Analysis

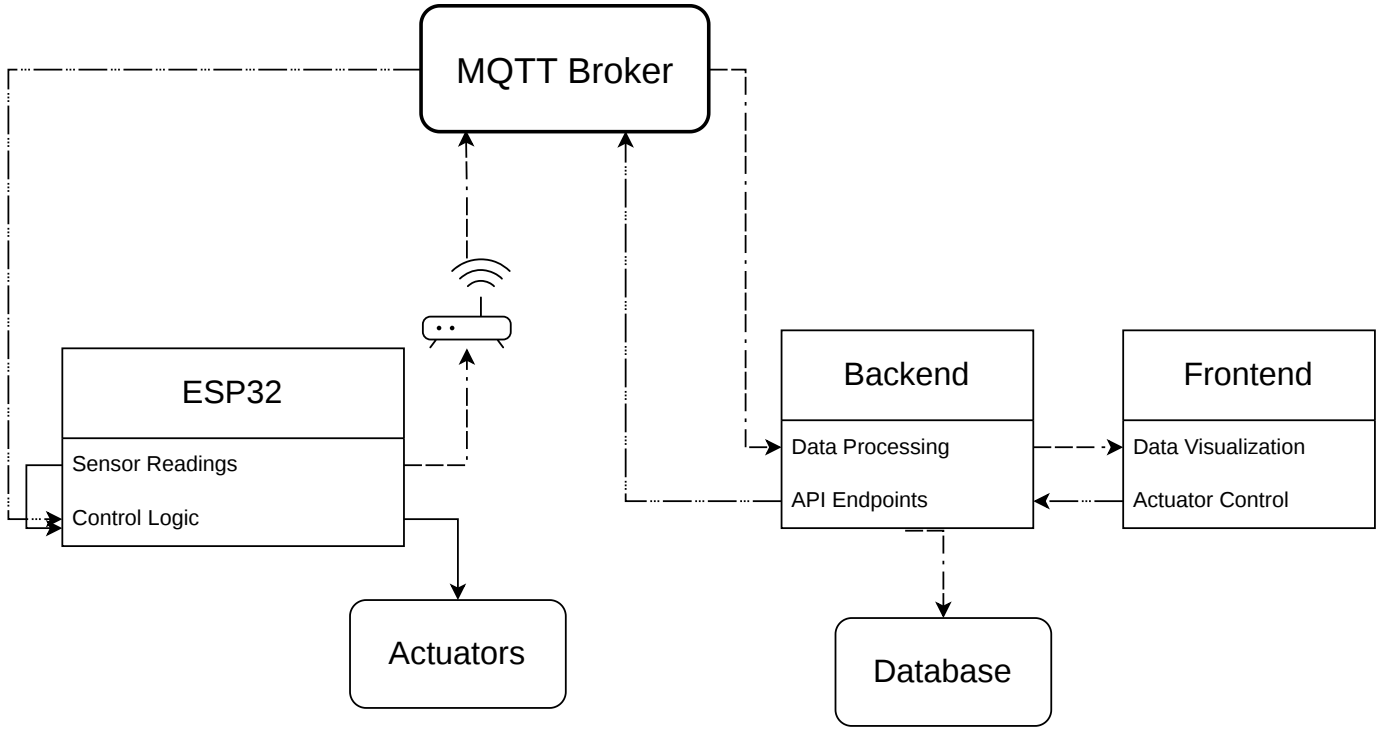
All system activity is sent to a backend and stored in a database using:

- structured sensor data
- actuator states

This enables:

- historical analysis
- optimization of plant care
- future scalability (AI-based decisions)

2 System Architecture



3 Layered IoT architecture Mappings

3.1 Device Layer

The Device Layer consists of all physical sensors and actuators connected to the system. These include two soil moisture sensors for monitoring different zones, a temperature and humidity sensor, a light sensor, a water level sensor, and a flame sensor for safety detection. On the output side, actuators such as a water pump, fan, servo motor, stepper motor, and buzzer are used to interact with the environment. These components are directly interfaced with the ESP32, which serves as the central hardware controller.

3.2 Edge Layer

The Edge Layer is implemented using the ESP32, which is a low-cost microcontroller with built-in WiFi and Bluetooth capabilities. It is based on a dual-core processor and includes integrated peripherals such as analog-to-digital converters (ADC), digital input/output pins (GPIO), timers, PWM controllers, and communication interfaces. These features allow it to interact directly with sensors and actuators while also handling wireless communication.

The ESP32 reads analog signals from sensors using its ADC, which converts continuous voltage levels into digital values that can be processed in software. Digital sensors communicate using protocols or simple HIGH/LOW signals. For output, the ESP32 can generate digital signals to control devices such as relays or transistors, and PWM signals to control components like servo motors. Its internal processing unit executes programmed logic in real time, allowing it to respond quickly to changes in sensor input. In the context of this project for example, if soil moisture falls below a certain level, it determines which zone requires watering and activates the stepper motor to position the pipe accordingly, followed by activating the water pump.

The ESP32 also manages communication with the rest of the system. It connects to a WiFi network and uses the MQTT protocol to publish sensor data and actuator states to the backend. At the same time, it subscribes to control messages, allowing it to receive commands from the user through the application. These commands can override automatic behavior, enabling manual control of actuators.

3.3 Network Layer

The Network Layer handles communication between the ESP32 and the backend using WiFi and the MQTT protocol, managed by the Mosquitto.

MQTT works on a publish/subscribe model. The ESP32 publishes sensor data to a topic (e.g., smartplant/data), and the backend subscribes to receive it. The backend can also publish control commands to another topic (smartplant/control), which the ESP32 subscribes to. The broker acts as a middleman, routing messages between devices without direct connections.

In addition to communication, the backend can process incoming data, such as analyzing trends or predicting future watering needs based on previous sensor readings.

3.4 Application Layer

Finally, the Application Layer provides the user interface through a web application built with React. This interface allows users to monitor real-time sensor data, control actuators remotely, and adjust system settings such as thresholds or operation modes. The application communicates with the backend using HTTP requests, making the system accessible from any device with a web browser.

4 Hardware Design and Components

The hardware design of the system is centered around the ESP32, which acts as the main controller and interface between all sensors and actuators. All components are connected to the ESP32 using a combination of analog input pins, digital input/output pins, and PWM-capable pins.

The system uses **two capacitive soil moisture sensors**. These sensors work by measuring the dielectric constant of the soil. Water has a higher dielectric constant than dry soil, so when moisture increases, the capacitance of the sensor changes. This change is converted into an analog voltage signal. The ESP32 reads this voltage using its analog-to-digital converter (ADC). In the system, when the measured value indicates dry soil, the pump is activated and the stepper motor rotates the pipe toward that zone.

A **DHT11** is used to measure temperature and humidity. It contains a thermistor for temperature measurement and a capacitive humidity sensor. The humidity sensor changes capacitance based on the amount of water vapor

in the air, while the thermistor changes resistance with temperature. These signals are processed internally and sent as a digital output. In the system, high temperature readings trigger the fan and slightly increase the watering duration.

A **photoresistor** (LDR) is used to measure light intensity. It works based on the photoconductive effect: its resistance decreases when light intensity increases. This change in resistance is converted into a voltage using a voltage divider circuit, which is then read by the ESP32 as an analog value. In the system, when light exceeds a threshold, the servo motor is activated to open a cover.

A **water level sensor module** operates using exposed conductive traces. When water comes into contact with the traces, it allows current to flow between them. The amount of current depends on how much of the sensor is submerged, producing an analog signal. In the system, if the water level falls below a threshold, watering is prevented and the system can generate an alert.

A **flame sensor module** typically uses an infrared (IR) photodiode or phototransistor that is sensitive to wavelengths emitted by flames (around 760–1100 nm). When a flame is detected, the sensor outputs a digital signal. In the system, this immediately activates the buzzer as a safety alert.

For actuation, a **DC water pump** is used to move water. It operates using an electric motor that converts electrical energy into mechanical rotation, which drives an impeller to push water through a pipe. The ESP32 controls the pump via a transistor or relay by switching the power supply on or off. The pump is activated when soil moisture readings indicate dryness.

A **DC fan** works similarly, using a motor to rotate blades and move air. It is powered through a control circuit and turned on when the temperature sensor detects high temperature.

A **servo moto**r (SG90) operates using pulse-width modulation (PWM). It contains an internal control circuit and feedback system that positions the shaft at a specific angle based on the width of the input pulse. In the system, it adjusts the position of a cover depending on light intensity.

A **stepper motor** (28BYJ-48 with ULN2003 driver) moves in discrete steps. It works by energizing coils in a specific sequence to create a rotating magnetic field, which turns the motor shaft in precise increments. In the system, it rotates the watering pipe to align with the soil zone that requires water.

Finally, a **buzzer module** converts electrical signals into sound using

a piezoelectric element. When voltage is applied, the material vibrates and produces sound waves. In the system, it is activated when the flame sensor detects fire.

For the actuators, additional driver components are required because the ESP32 cannot supply enough current directly. The DC water pump is controlled using transistor-based switching circuits or relay modules. These allow the ESP32 to safely turn higher-power devices on and off using low-power control signals. The stepper motor (28BYJ-48) is connected through a ULN2003 driver module, which handles the current required by the motor coils and allows the ESP32 to control it using multiple digital pins.

All components share a common ground to ensure stable operation. Power is supplied either through an external power source (4x1.5V battery pack) for the pump and the two motors.

5 Software Architecture

The software architecture of the system is divided into three main components: embedded firmware running on the ESP32, a backend server built with Node.js and Express, and a web application developed using React. These components communicate through MQTT and HTTP to provide both automation and user control.

At the edge level, the ESP32 runs firmware written in C/C++ using the Arduino framework. The firmware is modular, with separate files for configuration (config.h), control logic (logic.h), and the main execution loop. It is responsible for reading sensor data, applying automatic control logic, and controlling actuators. It also handles MQTT communication by publishing sensor and actuator data to the smartplant/data topic and subscribing to the smartplant/control topic to receive commands. The firmware supports both automatic and manual modes, allowing external commands to override local logic when needed.

The backend is implemented using Node.js with the Express framework. It acts as the central processing unit of the system. It connects to the MQTT broker (Mosquitto) to receive real-time data from the ESP32 and to send control commands. Incoming data is parsed and stored in a MongoDB database. The backend also exposes REST API endpoints that allow the frontend to send commands and retrieve data. In addition to basic data handling, the backend can perform analysis on historical data, such as identifying pat-

terns or predicting future watering needs. It also tracks actuator states and whether actions were triggered automatically or manually.

The frontend is a web application built with React. It provides a user interface for monitoring and controlling the system. The application communicates with the backend through HTTP requests, allowing users to send commands (e.g., turning the pump or fan on/off, switching modes, adjusting thresholds) and view system data. This interface enables real-time interaction with the system from any device with a web browser.

6 Communication Protocol Description

The system uses two communication protocols: MQTT for real-time communication between the edge device and the backend, and HTTP for interaction between the frontend and the backend.

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol designed for resource-constrained devices and unreliable networks. It operates on a publish/subscribe model, where clients communicate through a central broker instead of directly with each other. In this system, the Mosquitto is used to manage message exchange.

Each device connected to the broker acts as a client. The ESP32 functions as both a publisher and a subscriber. It publishes sensor readings and actuator states to the topic smartplant/data at regular intervals. The backend subscribes to this topic to receive real-time data. The data is formatted as a JSON object containing sensor values, actuator states, and metadata such as whether actions were triggered automatically or manually.

For control, the backend publishes messages to the topic smartplant/control. The ESP32 subscribes to this topic and processes incoming commands. These commands can include actions such as turning actuators on or off, switching between automatic and manual modes, or updating threshold values. This bidirectional communication enables full remote control of the system.

MQTT also supports asynchronous communication, meaning the ESP32 and backend do not need to be active at the exact same time to exchange messages. The broker handles message distribution, which simplifies system design and improves scalability. Additionally, MQTT is efficient in terms of bandwidth and power usage, making it suitable for embedded systems.

For user interaction, the frontend communicates with the backend using HTTP. The web application built with React sends RESTful API requests

to the backend server. For example, when a user presses a control button, the frontend sends a POST request containing a command. The backend then converts this request into an MQTT message and publishes it to the appropriate topic. Similarly, the frontend can request stored data using GET requests, which the backend retrieves from the database.

7 Database Schema

The system uses a NoSQL database implemented with MongoDB and managed through Mongoose in the backend. The schema is designed to store both continuous system logs and specific watering events.

The main collection, Log, is used to store periodic snapshots of the system. Each document represents the state of the system at a specific time. It includes a timestamp, sensor readings, and actuator states. Sensor data is grouped into a nested object containing values such as soil moisture (for two zones), temperature, humidity, light intensity, and water level. Actuator data is also grouped, storing the state of each actuator (on/off), along with a "source" field indicating whether the action was triggered automatically or manually. Additional fields such as servo angle and active zone are included to capture system behavior more precisely.

This structure allows the system to track not only environmental conditions but also how the system responded at that exact moment. This is important for debugging, monitoring, and future data analysis.

A second collection, WateringEvent, is used to store specific irrigation actions. Unlike the Log collection, which records continuous data, this collection records discrete events. Each document includes a timestamp, the zone that was watered, and the trigger type (manual or automatic). It also stores sensor values recorded just before watering, such as soil moisture and water level. This allows analysis of why watering occurred and how effective it was. An optional notes field can be used to store additional context.

The separation of these two collections improves clarity and efficiency. The Log collection provides a continuous stream of system data, while the WateringEvent collection focuses on important actions. Together, they support both real-time monitoring and long-term analysis, such as identifying watering patterns or optimizing thresholds.

```
Log
  timestamp
  sensors
      soil1
      soil2
      temperature
      humidity
      light
      waterLevel
  actuators
       pump (state, source)
       fan (state, source)
       buzzer (state, source)
       servoAngle
       zone

WateringEvent
  timestamp
  zone (1 or 2)
  trigger (auto/manual)
  sensorsBefore
      soil1
      soil2
      waterLevel
  notes
```

8 Iot Data Lifecycle

8.1 Data Generation

Data is generated at the physical environment through sensors embedded in the system. Each sensor continuously observes a specific environmental parameter. Soil moisture sensors produce analog signals based on soil conductivity, temperature and humidity sensors generate calibrated digital readings, and light and water level sensors produce signals proportional to environmental exposure and resource availability. Additionally, safety-related sensors generate event-based signals when abnormal conditions are detected.

8.2 Data Acquisition

The ESP32 is responsible for acquiring raw sensor data from all connected devices. It periodically samples analog inputs through its ADC channels and reads digital sensor outputs using GPIO or communication protocols. During this stage, raw signals are converted into structured digital values and temporarily held in memory for immediate processing.

8.3 Data Transmission

Once acquired, data is packaged into structured messages (typically JSON format) and transmitted over a wireless network. The system uses a lightweight messaging approach where sensor updates and system states are sent in discrete packets. This ensures efficient communication even under constrained bandwidth conditions. Control messages follow the same structure in the opposite direction, enabling bidirectional interaction between the system components.

8.4 Data Storage

All transmitted data is persisted in a centralized database. The storage layer organizes information into time-indexed records that capture both environmental conditions and system behavior. This allows the system to maintain a historical record of sensor readings and actuator states, enabling traceability and long-term analysis. Data is stored in a flexible schema to accommodate varying sensor inputs and evolving system requirements.

8.5 Data Processing

Data processing occurs primarily at the backend layer. Incoming sensor values are validated, cleaned, and structured before being stored. The system may also perform aggregation over time windows, such as averaging environmental conditions or detecting trends. In addition, rule-based logic can be applied to identify conditions that require action, such as thresholds for irrigation or temperature-based responses. This stage transforms raw data into actionable information.

8.6 Data Visualization

Processed and stored data is made accessible through the web interface. The visualization layer presents real-time and historical information in an intuitive format, allowing users to observe environmental trends and system behavior over time. This includes graphical representations of sensor evolution and status indicators for system components, enabling easier interpretation of complex data patterns.

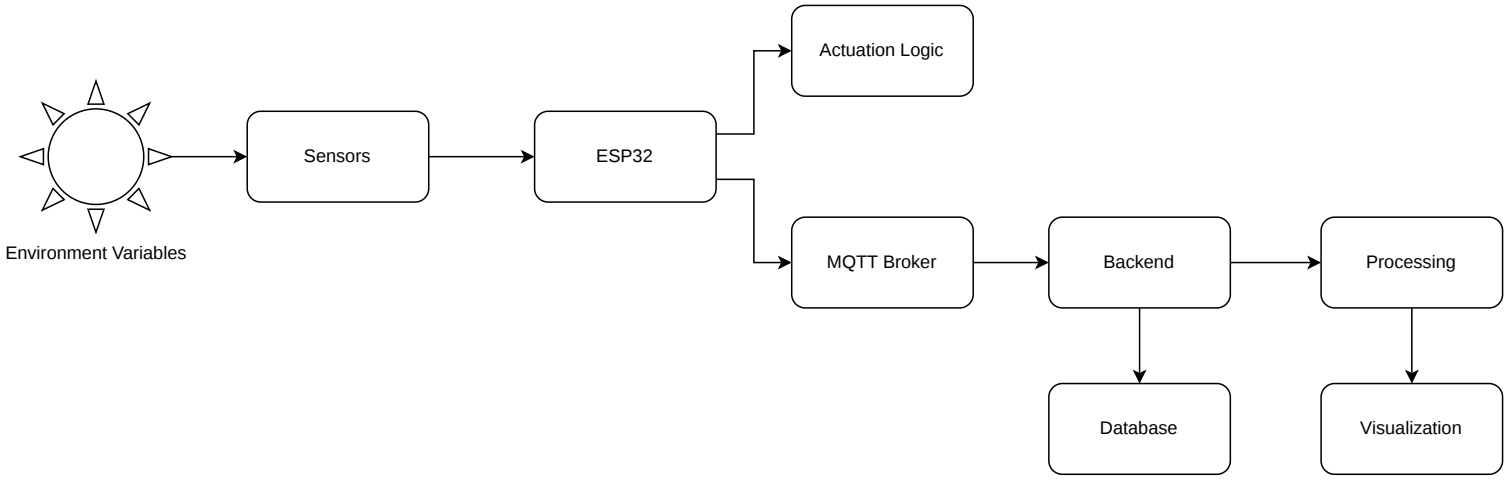
8.7 Data Actuation

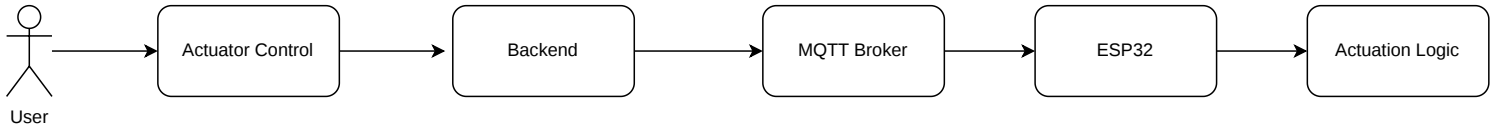
Actuation represents the decision execution stage of the lifecycle. Based on processed data or direct user input, control signals are generated and sent back to the edge device. These signals trigger physical responses such as activating irrigation, adjusting ventilation, or changing mechanical positioning systems. Actuation can be either automated, based on predefined logic, or manual, initiated by user commands.

8.8 Data Retention Strategy

The system follows a structured retention approach to balance storage efficiency and data utility. Recent high-resolution data is retained to support real-time monitoring and short-term analysis, while older data can be summarized or aggregated to reduce storage overhead. Critical events, such as irrigation actions or abnormal sensor readings, are preserved for longer periods due to their analytical importance. This tiered retention strategy ensures efficient database usage while maintaining access to meaningful historical insights

9 Data Flow Diagram





10 Backend Analysis

10.1 Incoming Telemetry Flow

Sensor data generated at the device level is transmitted through MQTT and received by the backend message handler. Each incoming message is parsed and immediately transformed into a structured telemetry object containing sensor readings and system metadata.

Once received, the data follows two parallel paths:

- It is stored persistently in the database as a new log entry for historical tracking.
- It is cached in memory as the latest system state for fast retrieval without database access.

This dual-path approach ensures both real-time responsiveness and long-term persistence.

10.2 Real-Time State Retrieval

When a client requests the latest system state, the backend first attempts to return the most recent in-memory telemetry snapshot. This avoids unnecessary database queries and reduces latency. If no cached data exists, the system falls back to retrieving the latest record from persistent storage.

10.3 Control Command Flow

User actions from the frontend are translated into structured control commands by the backend. These commands are not executed directly by the server; instead, they are published to the messaging layer.

The flow is as follows:

1. User request is received via HTTP endpoint
2. Backend validates input parameters
3. A control command string is generated
4. Command is published to the control topic

The edge device then interprets and executes the command. This design ensures loose coupling between user interaction and physical actuation.

10.4 Watering Event Tracking

When irrigation is manually triggered, the backend captures a snapshot of current environmental conditions before issuing the command. This snapshot is stored as a discrete event entry.

Each watering event includes:

- Target irrigation zone
- Trigger source (manual or automated)
- Sensor values prior to actuation

This allows later analysis of irrigation decisions and environmental conditions at the time of action.

10.5 Predictive Analysis: Watering Forecasting

The system includes a lightweight predictive module that estimates the next irrigation time using historical sensor data and environmental behavior patterns. The goal of this module is not to train a complex machine learning model, but to derive a practical estimation based on observed soil moisture dynamics.

The prediction process relies on time-series analysis of recent logs. The backend retrieves a window of past sensor readings and reconstructs the evolution of soil moisture over time. From this sequence, it computes the average moisture level and the rate of change between consecutive readings.

The core factor used in prediction is the **soil drying rate**, which represents how quickly moisture decreases over time. This is calculated by measuring the difference in soil moisture between consecutive timestamps and normalizing it by the time interval between them. Only valid time intervals are considered to avoid distortions caused by irregular updates.

Once the average drying rate is determined, the system estimates how long it will take for the soil to reach a predefined irrigation threshold. This threshold represents the moisture level at which watering is typically triggered. The time remaining is computed by dividing the difference between the current average moisture and the threshold by the estimated drying rate.

To improve robustness, the prediction is adjusted using contextual environmental factors:

- **Temperature influence:** Higher temperatures increase evaporation, so the predicted time is reduced when ambient temperature exceeds a defined range.
- **Recent irrigation history:** If watering occurred recently, the system prevents unrealistic immediate rewatering by enforcing a minimum delay window.
- **Data stability check:** Predictions are only considered reliable when enough historical data points exist; otherwise, confidence is reduced.

The final output includes an estimated timestamp for the next expected irrigation event, the number of hours remaining, and a confidence level indicating the reliability of the prediction. This confidence is derived from the consistency of historical drying patterns and the amount of available data.

10.6 Data Validation and Fault Handling

Incoming data is validated before processing to ensure structural integrity. If parsing fails or malformed messages are received, they are safely ignored without disrupting system execution.

11 Testing and Validation

The Smart Plant System was validated through both long-term real-world operation and controlled forced testing scenarios to ensure reliability, stability, and correct system behavior under varying conditions.

11.1 Long-Term Operational Testing

The system was deployed and run continuously over multiple days in a realistic environment to observe its behavior under natural environmental variations. During this period, sensor readings were collected at regular intervals and logged into the system database, while automated decisions were executed in real time.

This extended testing phase allowed us to evaluate:

- consistency of sensor readings over time

- stability of MQTT communication between components
- correctness of automated actuation events (watering, ventilation, and safety responses)
- responsiveness of the system to gradual environmental changes

Over time, the system demonstrated stable behavior, correctly triggering automated actions when environmental thresholds were reached and maintaining proper synchronization between sensing, decision-making, and actuation processes.

11.2 Forced and Edge Case Testing

In addition to long-term observation, the system was subjected to forced testing scenarios to evaluate robustness under abnormal or extreme conditions. These tests were designed to simulate edge cases that may occur in real deployment.

Examples of forced tests include:

- manually setting sensor values to extreme high and low ranges
- triggering rapid successive watering commands
- simulating sudden loss or delay of sensor data transmission
- testing actuator commands in conflicting states (e.g., repeated ON/OFF toggles)
- disconnecting and reconnecting network communication to test recovery behavior

These tests helped validate that the system can safely handle unexpected inputs without crashing or entering undefined states. Validation logic in the backend and structured command handling in the communication layer ensured that invalid operations were rejected or safely ignored.

11.3 Physical Deployment Evaluation

During implementation, multiple hardware configurations and setups were tested before finalizing the design. Different placements and wiring arrangements of sensors and actuators were evaluated to ensure stable readings and reliable operation.

Through this iterative process, it became clear that the system functions effectively as a compact and modular prototype representing a larger-scale smart farming or homestead automation network. The final setup can be interpreted as a scaled-down model of a fully integrated IoT-based agricultural monitoring system.

This validation phase confirmed that the system is not only functionally correct but also structurally adaptable for future expansion into larger deployments.

12 Work Partition

The development of the Smart Plant System was carried out collaboratively, with tasks divided between both team members while maintaining shared responsibility for hardware integration and system debugging.

Both team members contributed to the hardware assembly and firmware development, including sensor integration, actuator wiring, and ESP32 programming. This joint effort ensured that the physical system operated correctly as a unified prototype.

Marc Alwan focused primarily on actuator control logic, including implementing and testing the control of the pump, fan, servo motor, stepper motor, and safety components. He also contributed to backend improvements related to MQTT communication and developed the predictive watering function used for estimating future irrigation needs based on historical sensor data.

Hasan Ramadan was responsible for implementing the real-time data visualization and user interface, including live sensor readings, dashboards, and control panels in the web application. He also set up the main backend infrastructure, including the Node.js server MQTT broker and database integration, ensuring reliable storage and retrieval of system logs and events.

13 Security Considerations

he system does not handle sensitive or personal data, as it is limited to environmental sensor readings and actuator states. Therefore, security risks related to data privacy are minimal.

The main concern is ensuring that only valid control commands are processed to avoid unintended actuator behavior. This is handled through basic input validation and restricted command formats at the backend level.

Given that the system is a prototype for a controlled IoT environment, advanced security mechanisms such as encryption and authentication were not implemented as they were not essential for the scope of this project.

14 Challenges and Lessons Learned

14.1 Challenges

During the development of the Smart Plant System, several practical and external challenges affected the implementation process. Due to external circumstances related to the war situation, team meetings and coordinated on-site work were significantly limited. This led to delays in planning, integration, and testing phases.

Additionally, hardware procurement was rushed due to time constraints and supply limitations. As a result, some components were not fully verified before integration, which led to cases of malfunctioning hardware, including a few components that were damaged or burned during early testing. These issues required repeated replacement and troubleshooting, which slowed down development progress.

Another challenge was achieving stable integration between all system components under time pressure, especially while simultaneously debugging hardware, firmware, and network communication layers.

14.2 Lessons Learned

From an Internet of Things perspective, this project highlighted the importance of incremental integration and thorough hardware validation before full system assembly. Rushing component assembly can lead to cascading failures that are harder to debug at later stages. We also learned the im-

portance of designing fault-tolerant communication systems, especially when dealing with MQTT-based distributed architectures.

From a networking perspective, we gained practical understanding of real-time communication constraints, including message reliability, latency handling, and the importance of persistent message structures in unstable environments.

From an agricultural systems perspective, the project reinforced how environmental monitoring must be adaptable and robust, since real-world conditions are highly variable and sensors alone cannot fully capture all external influences. It also highlighted the value of automation in reducing manual intervention, while still requiring careful calibration to avoid overwatering or under-reaction.

Overall, the challenges encountered helped strengthen both our technical problem-solving skills and our understanding of how IoT systems behave under real deployment constraints.